第三章 SQL 注入漏洞与防护——实验分析报告

任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关

因为很多网页功能（如用户信息查询、商品搜索、登录验证）都需要从后台数据库中动态获取数据。页面本身只是一个静态展示框架，无法脱离数据库独立生成个性化内容。只有根据用户输入的特定条件（例如“用户 ID”“商品关键词”），才能从数据库中筛选出对应的数据并展示给用户。因此，用户输入必然会与数据库查询产生直接关联。

具体业务场景：

以本实验使用的 DVWA（Damn Vulnerable Web Application）SQL Injection 模块为例。该页面提供一个输入框，要求用户输入 User ID，点击提交后页面会显示该用户的 First name 和 Surname。页面显示的内容完全依赖于用户输入的 ID 从数据库中查询得到。

2.一个最小业务链路是什么

用户输入 → 查询条件 → 数据库执行 → 返回结果

对应 DVWA 场景的实例（分步说明）：

用户输入：用户在页面表单中提交数据，例如输入 User ID 为 1。

查询条件：后端代码将输入拼接到 SQL 语句的 WHERE 子句中，形成完整的查询条件字符串 WHERE user_id = '1'（此时尚未发给数据库）。

数据库执行：数据库服务器接收到完整 SQL 语句 SELECT first_name, last_name FROM users WHERE user_id = '1'，进行语法解析并执行。

返回结果：数据库将结果集返回给后端，后端渲染到页面，显示 First name: admin, Surname: admin。

3. 用户输入是如何进入查询条件的

以 DVWA Low 级别的后端核心代码（PHP）为例：

```
$id = $_GET['id'];     // 从 HTTP 请求中获取用户输入
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
```

具体过程：

用户在浏览器输入框输入 User ID（例如 1），点击提交。

浏览器构造 HTTP GET 请求（例如 http://localhost/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit）。

服务器端 PHP 脚本通过 $_GET['id'] 获取到字符串 "1"。

程序将该字符串直接拼接到 SQL 语句模板中，替换掉 '$id' 的位置。

拼接后得到的完整 SQL 查询字符串被发送给 MySQL 数据库执行。

4. 为什么这为 SQL 注入创造了可能性

根本原因：用户输入被当成了 SQL 代码结构的一部分来解析，而不是纯粹的数据值。攻击者可以在输入中嵌入 SQL 的特殊字符和关键字，从而改变查询条件的逻辑结构。

具体解释：

正常输入：用户输入 1 → 查询条件变为 WHERE user_id = '1' → 数据库只返回 ID 为 1 的用户。

恶意输入：攻击者输入 1' OR '1'='1 → 拼接后查询条件变为 WHERE user_id = '1' OR '1'='1'。单引号 ' 提前闭合了原字符串，OR '1'='1' 引入了一个永远为真的逻辑。整体条件恒为真，导致数据库返回表中所有用户的记录，而不仅仅是 ID 为 1 的用户。

关键认知：不是因为输入中包含了特殊字符（例如单引号 '）才危险。假如用户输入 O'Brian 作为姓名，单引号本身无害。

危险在于：输入被放进不该放的位置——即 SQL 代码的拼接过程中，而不是作为参数绑定。一旦输入内容能够干预 SQL 的语法结构，注入就发生了。

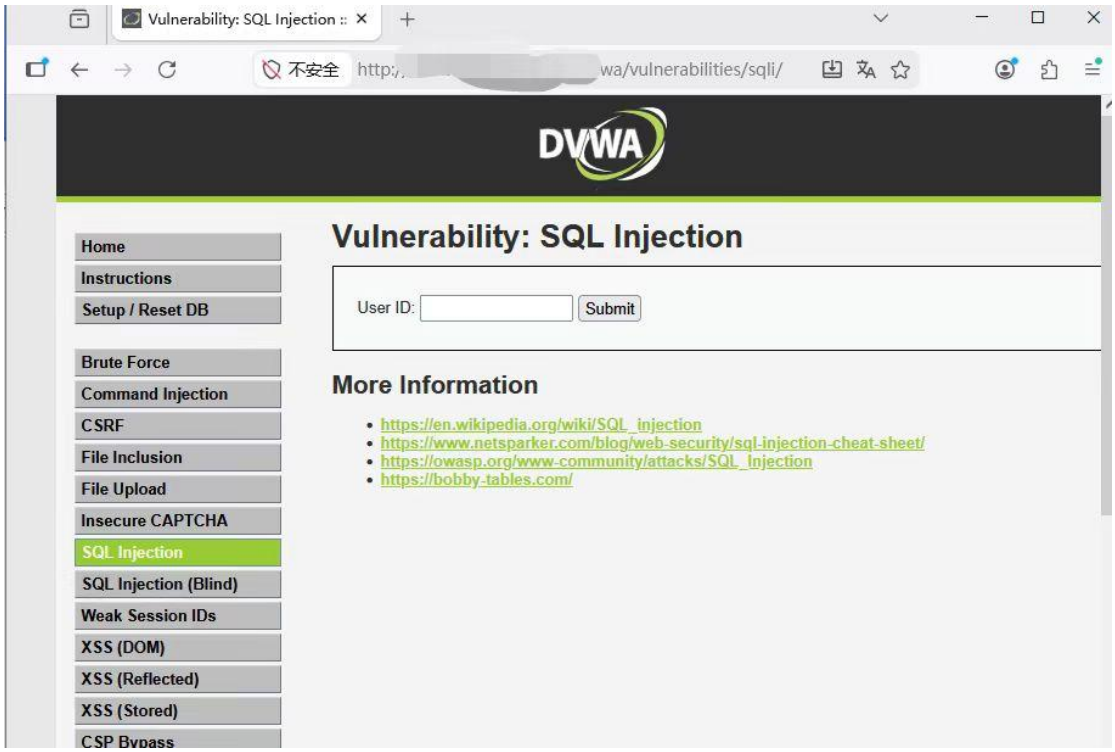
任务 2：DVWA SQL Injection Low 现象记录

1. 实验页面 / 实验目标

实验页面：DVWA 的 SQL Injection 模块。

页面功能：提供一个文本输入框，要求用户输入 User ID，点击 Submit 后，页面下方会显示该 ID 对应的用户 First name 和 Surname。

实验目标：观察普通输入与恶意注入输入在返回结果上的差异，并分析差异背后反映的 SQL 查询结构是否被改变。



分析判断：该页面正常功能是根据用户提供的 ID 从数据库中查询并显示一条对应的用户记录。如果后续输入导致返回多条记录或非预期内容，就说明查询逻辑被干预了。

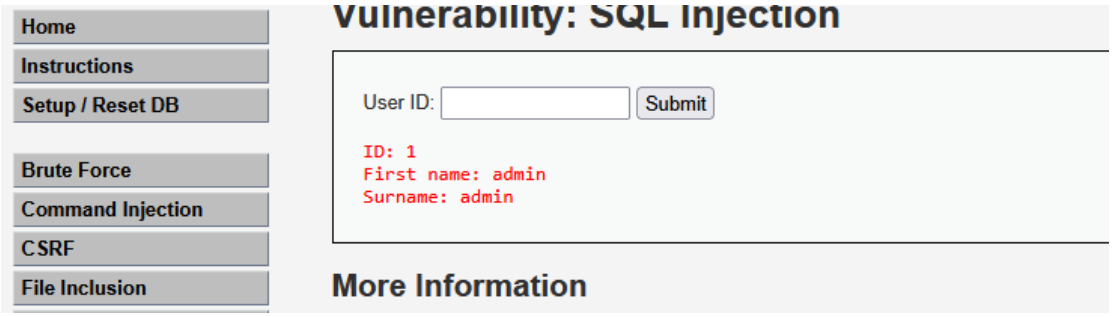
2. 普通输入与异常输入的对比

（1）普通输入

输入内容：1

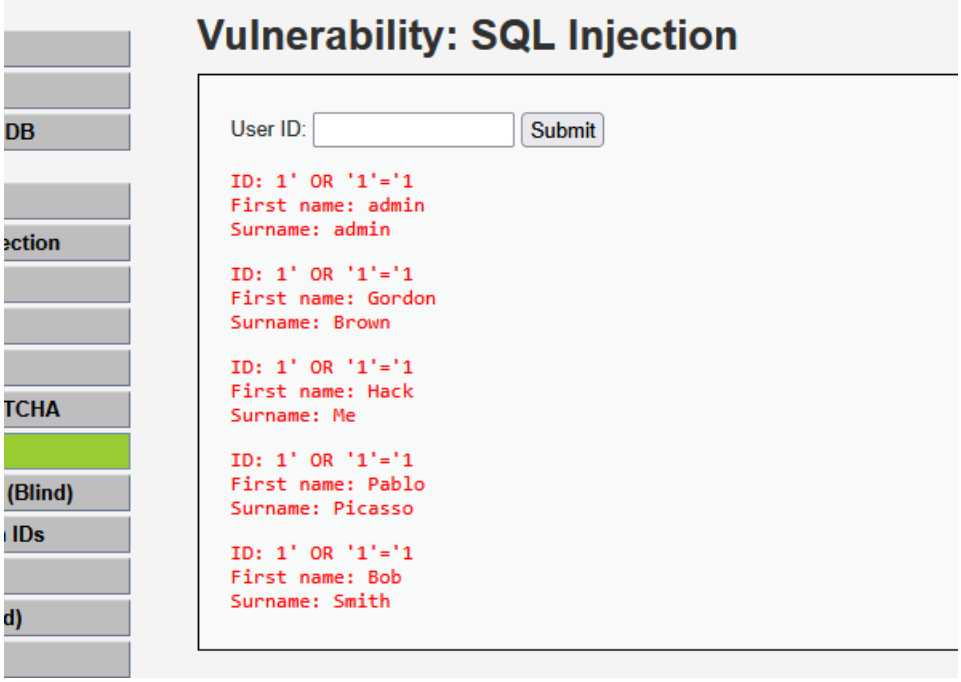
操作：在输入框中输入 1，点击 Submit。

返回结果：页面显示 First name: admin, Surname: admin。



分析判断：输入 1 返回的是 ID=1 的用户（admin）。这是符合页面设计预期的正常查询变化——不同 ID 会返回不同用户，但每次只返回一条。该截图作为后续对比的基准。

2）异常输入（注入 payload）
输入内容：1' OR '1'='1
操作：在输入框中输入 1' OR '1'='1，点击 Submit。
返回结果：页面显示 5 条用户记录：
admin、Gordon、Hack、Pablo、Bob 的 first name 和 surname。



分析判断：输入 1' OR '1'='1 后，页面返回了数据库中所有用户，而不仅仅是 ID=1 的用户。这个差异直接说明：输入内容中的 ' 和 OR '1'='1 被数据库解释为 SQL 语法，改变了 WHERE 子句的逻辑（从 user_id='1' 变为 user_id='1' OR '1'='1'，后者恒为真）。这个差异支撑的判断是：查询的“结构”被污染了，而不是仅仅改变了查询参数。
3. 返回结果的差异
普通输入 1：返回 1 条记录（仅 admin）。
异常输入 1' OR '1'='1：返回 5 条记录（所有用户）。
核心差异总结：
普通输入下，返回记录数量与输入值有严格对应关系（ID=1 → 一条；ID=2 → 另一条）。异常输入则导致返回了全部用户数据，这在正常业务中绝不可能发生——因为不存在一个合法的 User ID 能同时匹配所有用户。
差异支撑的判断：返回记录数量的“数量级变化”（1 条 → 全部）是查询逻辑被改变的铁证。如果只是参数不同，最多只改变返回哪一条记录，绝不会改变返回的记录总数。
4. 为什么这种差异不像“正常查询变化”
“正常查询变化”的定义：
用户输入不同的合法 ID（如 1、2、3），页面会返回对应的不同单条用户记录。变化的仅仅是“返回哪一条数据”，返回记录的数量性质始终是 1 条。
本次差异与正常变化的区别：
输入 1 返回 1 条 → 正常。
输入 2 返回 1 条（Gordon） → 正常（参数变化，结构没变）。
输入 1' OR '1'='1 返回 5 条 → 不正常。
为什么不像正常变化：

因为正常变化永远不会改变 WHERE 子句的逻辑结构，只会改变比较的具体值。而注入输入 1' OR '1'='1 通过引入 SQL 关键字 OR 和恒等条件 '1'='1'，重写了 WHERE 子句的语义——从“精确匹配 ID”变成了“无条件为真”。这种结构性的改变，不可能由任何合法的用户 ID 输入产生。

最终判断：

该差异支撑的结论是 —— 输入内容没有被当作普通数据，而是被拼接到了 SQL 语句中并改变了语法结构。这正是 SQL 注入漏洞存在的直接证据，也是区分“普通查询变化”与“注入攻击”的关键标志。

任务 3：机制分析与差异比较

1. 输入如何进入查询结构

以 DVWA Low 级别的核心代码为例：

```
$id = $_GET['id'];
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = mysqli_query($connection, $query);
```

完整链条如下：

输入：用户在浏览器输入 1' OR '1'='1，提交 HTTP GET 请求。

拼接：PHP 使用 $_GET['id'] 获取该字符串，直接嵌入到 SQL 语句模板的 '$id' 位置。

解析：MySQL 数据库接收到拼接后的完整 SQL 语句：

SELECT first_name, last_name FROM users WHERE user_id = '1' OR '1'='1'

执行：数据库解析该语句，发现 WHERE 子句中的 OR '1'='1' 恒为真，因此扫描全表返回所有用户记录。

回显：后端将查询结果渲染到 HTML 页面，用户看到所有用户的姓名。

关键节点：拼接这一步是结构和数据混淆的根源。若此处使用参数化查询（预编译），输入将永远作为数据而非代码。

2. 为什么根因是“结构被污染”而不是“字符本身危险”

反例证明：假设输入中包含单引号 ' 但使用安全的参数化查询：

```
$stmt = $conn->prepare("SELECT ... WHERE user_id = ?");
$stmt->bind_param("s", $id);
$stmt->execute();
```

此时即使用户输入 1' OR '1'='1，数据库只会将其作为一个完整的字符串值去匹配 user_id 字段，而不会解释其中的 OR 关键字。单引号不会造成任何危害。

正例说明：在拼接场景下，即使对输入做了“转义”（如 Medium 级别使用 mysqli_real_escape_string），仍可能存在数字型注入，攻击者不需要使用任何引号：

输入：1 OR 1=1

拼接后：WHERE user_id = 1 OR 1=1

这里完全没有单引号，但仍然改变了查询结构（通过 OR 1=1）。

结论：

字符本身（如 '、;、--）只是攻击的“工具”，不是原因。

根因是“输入被拼接到 SQL 语句结构内部”，使输入内容能够干预语法解析。

强制判断题 1. SQL 注入的根因更接近

A. 输入里有几个危险字符

B. 输入进入了不该进入的查询结构

选 B。理由：如果只是危险字符问题，转义所有字符就能根治；但数字型注入无需字符，结构污染才是本质。

3. Low 与 Medium 的差异

Low 级别：无任何防护措施，直接拼接用户输入。因此无论字符型注入 1' OR '1'='1 还是数字型注入 1 OR 1=1 都能成功。

Medium 级别：采用 mysqli_real_escape_string() 转义特殊字符，并将输入放在下拉菜单中（但可前端绕过）。

对于字符型注入：转义使单引号变为 \'，无法闭合原字符串，因此 1' OR '1'='1 此类 payload 失效。

对于数字型注入：转义不影响数字和操作符，攻击者输入 1 OR 1=1 依然可以成功（无需引号）。此外，即使使用下拉菜单，攻击者也可通过修改 HTTP 请求（如 ?id=1 OR 1=1）绕过限制。

核心差异总结：Medium 只能防御依赖引号闭合的注入，无法防御数字上下文中的逻辑注入（因为根因"结构污染"没有被消除，只是阻挡了部分攻击路径）。真正的防护必须使用参数化查询，而不是依赖转义或输入格式限制。

4. 普通 SQL 注入与 Blind SQL Injection 的差异

普通 SQL 注入：

页面会直接回显查询结果（如用户名、数据列）。

攻击者可以通过 UNION SELECT 直接读取数据库内容，效率高。

典型场景：用户信息展示、商品搜索列表。

盲注（Blind SQL Injection）：

页面不直接回显数据，只返回"成功/失败"、重定向、空白页或固定状态码。

攻击者需要通过构造布尔条件（AND 1=1 页面正常，AND 1=2 页面错误）逐个字符推断数据，或使用时间延迟（AND SLEEP(5)）观察响应时间来提取信息。

效率较低，但依然能窃取数据。

典型场景：登录验证接口、注册检查（是否已存在）、API 返回固定 JSON。

示例对比：

普通注入：输入 1' UNION SELECT username, password FROM users --，页面直接显示账号密码。

盲注：输入 1' AND ASCII(SUBSTRING(database(),1,1)) > 100 --，页面正常（true）或报错（false），攻击者据此二分猜解。

5. 为什么盲注和普通注入根因相同，但证据形式不同

根因相同：两种注入的底层原理完全一样——都是将恶意输入拼接到查询结构中，改变了 SQL 语句的原始逻辑。

普通注入常用恒真条件（OR '1'='1'）导致返回所有数据。

盲注常用恒假条件（AND 1=2）导致无数据返回，页面表现异常；或用时间延迟函数制造可观察的时差。

证据形式不同的根本原因：应用程序对查询结果的输出方式不同。

如果应用程序将查询结果直接渲染到页面（如显示用户名、商品列表），攻击者能看到具体数据 → 普通注入的证据是直接的数据回显。

如果应用程序只利用查询结果的布尔值来改变页面行为（如显示"登录成功"或"登录失败"），攻击者看不到具体数据，只能通过页面行为的差异（正确/错误、延迟时间）来推断 → 盲注的证据是行为差异或时间差。

一句话总结：

根因相同（结构污染），证据形式不同是因为输出侧的信息通道带宽不同——普通注入有"数据回显通道"，盲注只有"逻辑/时间通道"。

强制判断题 2. Blind SQL Injection 和普通 SQL 注入的最大差异更接近
A. 根因不同
B. 证据形式不同
选 B。理由：如果根因不同，那么防护方案应该不同，但实际上参数化查询对两者同样有效，说明根因相同。唯一差异是漏洞被利用时攻击者获取信息的途径不同，即证据形式不同。

任务 4：SQL 注入防护清单与解释

1. 参数化查询（预编译语句）
解决什么问题：从根本上杜绝 SQL 注入。它将 SQL 语句的“结构”与“数据”分离，用户输入永远只作为参数值绑定，不会被解释为 SQL 语法。即使输入包含 ' OR '1'='1，数据库也只会把它当作普通字符串去比较。
控制哪一层：根因控制。直接消除“输入被拼接到查询结构”这一根本原因。
不能替代什么：不能替代输入校验（如业务规则检查：年龄不能为负数），也不能替代输出编码或权限管理。参数化查询只保证 SQL 语句安全，不保证业务逻辑安全。另外，对于表名、列名、ORDER BY 后的排序字段等动态标识符，参数化查询无效（需用白名单校验）。
2. 输入校验
解决什么问题：在数据进入查询前，根据业务规则限制输入的格式、类型、长度或取值范围，从而减少攻击面。例如，要求 User ID 必须是数字，那么攻击者输入的 1' OR '1'='1 会因包含非数字字符而被拒绝。
控制哪一层：暴露面控制（也称“防御前置”）。它并不消除结构污染的根本原因，而是提前筛掉大部分恶意输入，降低漏洞被成功利用的概率。
不能替代什么：不能替代参数化查询。攻击者可利用允许的字符（如数字型注入 1 OR 1=1）绕过基于字符黑名单的校验；且校验规则可能被绕过（如 Unicode 欺骗、二次编码）。永远不要将输入校验作为唯一防线。
3. 错误处理
解决什么问题：防止数据库的详细错误信息（如 SQL 语法错误、表名、列名）泄露给客户端。攻击者通常利用错误信息来推断查询结构或数据库类型。自定义错误页面或返回统一错误消息（如“系统繁忙”）可以阻断这一信息渠道。
控制哪一层：后果范围控制（更接近“暴露面信息缩减”）。它不阻止注入发生，但大幅降低攻击者获取有价值信息的能力，使盲注难度增加。
不能替代什么：不能替代参数化查询。即使不显示错误，攻击者仍可通过盲注（时间差、布尔差异）逐位窃取数据。错误处理只是让攻击更困难，不是根治。
4. 最小权限原则
解决什么问题：限制数据库账号的权限，使注入成功后攻击者能造成的破坏最小化。例如，仅用于前端查询的账号只授予 SELECT 权限，不授予 INSERT、UPDATE、DROP 等。即使攻击者利用注入读取数据，也无法修改或删除表；更极端地，可针对不同查询使用不同账号（只读账号 vs 写账号）。
控制哪一层：后果范围控制。它不防止注入发生，但限制了攻击者“能做什么”，将损失控制在可接受范围。
不能替代什么：不能替代参数化查询。如果攻击者能通过注入读取敏感数据（如用户密码哈希），即使无法删表也是重大泄露。最小权限是深度防御的一环，必须与根因控制结合。

5. 其他合理措施：使用存储过程（谨慎）或 ORM 框架
解决什么问题：存储过程可以将 SQL 逻辑封装在数据库中，减少应用层拼接；但若存储过

程内部仍使用动态拼接，则同样存在注入风险。ORM（如 Entity Framework、Hibernate、Django ORM）通常默认使用参数化查询，从框架层面避免开发人员手动拼接 SQL。
控制哪一层：部分根因控制（如果使用参数化调用）或 无控制（如果存储过程内拼接）。正确使用的 ORM 可归为根因控制；不加分析的存储过程则只是转移风险。
不能替代什么：不能替代安全编码规范。开发者可能在 ORM 中回退到原生 SQL 拼接，或编写危险的存储过程。仍需强调参数化查询作为底线。

SQL 注入因果链/防护关系图

阶段	具体过程 / 现象	防护控制点（含控制层）
输入点	用户在 User ID 输入框中提交 1' OR '1'='1	
查询构造	直接拼接输入到 SQL 语句： "SELECT ... WHERE user_id = '$id'" 拼接后变为： SELECT ... WHERE user_id = '1' OR '1'='1'	参数化查询（根因控制：分离代码结构与数据，杜绝拼接）
数据库解析/执行	MySQL 接收到拼接后的 SQL：WHERE user_id='1' OR '1'='1'，解析时发现 OR '1'='1'恒真条件，执行全表扫描，返回 5 条记录	最小权限（后果范围控制：限制账号仅 SELECT，防止后续删表/修改）
结果差异/行为证据	页面本应返回 1 条用户记录，实际返回 全部 5 条记录（admin，Gordon，Hack，Pablo，Bob）	错误处理（后果范围控制：隐藏详细错误信息，提升盲注难度）
根因判断	结构被污染 —— 输入被拼接到查询结构中，成为 SQL 语法的一部分，而非仅仅作为数据值。根本原因不是特殊字符（如 '），而是输入进入了不该进入的代码结构。	
因果链总结	输入 → 拼接 → 解析 → 执行 → 行为差异，完整链条显示：恶意输入导致查询条件被改写，最终产生异常结果。	参数化查询直接切断“拼接”环节；最小权限和错误处理在后续环节限制攻击后果。

关系链与防护对应说明
因果链：输入点（恶意 payload） → 查询构造（拼接污染） → 数据库解析/执行（恒真条件） → 结果差异（返回全部数据） → 根因判断（结构污染）
防护对应：
参数化查询作用于查询构造阶段，属于根因控制（彻底避免结构污染）。
最小权限作用数据库执行阶段，属于后果范围控制（即使注入成功也无法删表）。
错误处理作用于结果差异阶段，属于后果范围控制（不暴露数据库细节，增加盲注难度）。

下一步改进计划
开发中强制使用参数化查询（PDO/prepared statements），绝不拼接用户输入。
测试时同时检查字符型注入和数字型注入，避免仅依赖转义。
生产环境配置最小权限数据库账户，关闭详细错误输出。

学生自评表

1.我本章最清楚的一点是什么？
我最清楚 SQL 注入的根本原因是查询结构被污染，而不是输入包含特殊字符；也理解了“输入→拼接→解析→执行→回显”的完整注入链路。
2.我最容易混淆的一点是什么？
容易混淆输入过滤和参数化查询的防护层级，一开始以为过滤字符就能防注入，后来明白参数化查询才是根因防护。
3.我本次作业中哪一段最需要教师复核？
普通 SQL 注入与盲注的差异部分，以及根因相同但证据形式不同的解释，希望老师确认理解是否准确。
4.如果重做一次，我会优先改哪一部分？
增加更多普通输入对比，以更清晰地证明“正常查询变化”只会改变返回哪一条数据，而不会改变记录数量；同时精简文字，内容更清晰。

AI 输出审核与人工复核记录
1. 本作业哪些内容使用过 AI 辅助
使用 AI 辅助梳理注入机制、防护清单、对比表格，帮我更好的完成报告。
2.哪些判断是你自己复核后保留的
SQL 注入根因是结构被污染
Low 与 Medium 级别防护差异
普通注入与盲注的证据形式不同
3.哪些内容你发现
AI 最初将“输入过滤、转义字符”标记为根因防护，我结合 DVWA 实验现象与课程核心知识点进行了修正：输入过滤与转义只能减少攻击面，无法从根本上阻止结构污染；只有参数化查询才是真正的根因控制。同时 AI 对数字型注入的解释不够清晰，我补充了“无引号也能注入”的关键说明，让机制更准确。
4.你最终如何确认结论和证据是一一对应的
我通过 DVWA 本地实验逐一验证：输入 1 返回单条记录为正常查询；输入 1' OR '1'='1 返回全部用户，证明查询结构被污染。所有结论均与页面返回结果、截图现象一一对应，确保实验真实、逻辑严谨。